

# Sign-Language Glove

Ayman EL-Hindi and Le Quoc Viet Pham

## PHASE 1

**Problem Definition:** A (soft) real-time embedded system to recognize signs and translate them into text or sounds, with minimal delay, to communicate effectively with people with disabilities and vice versa.

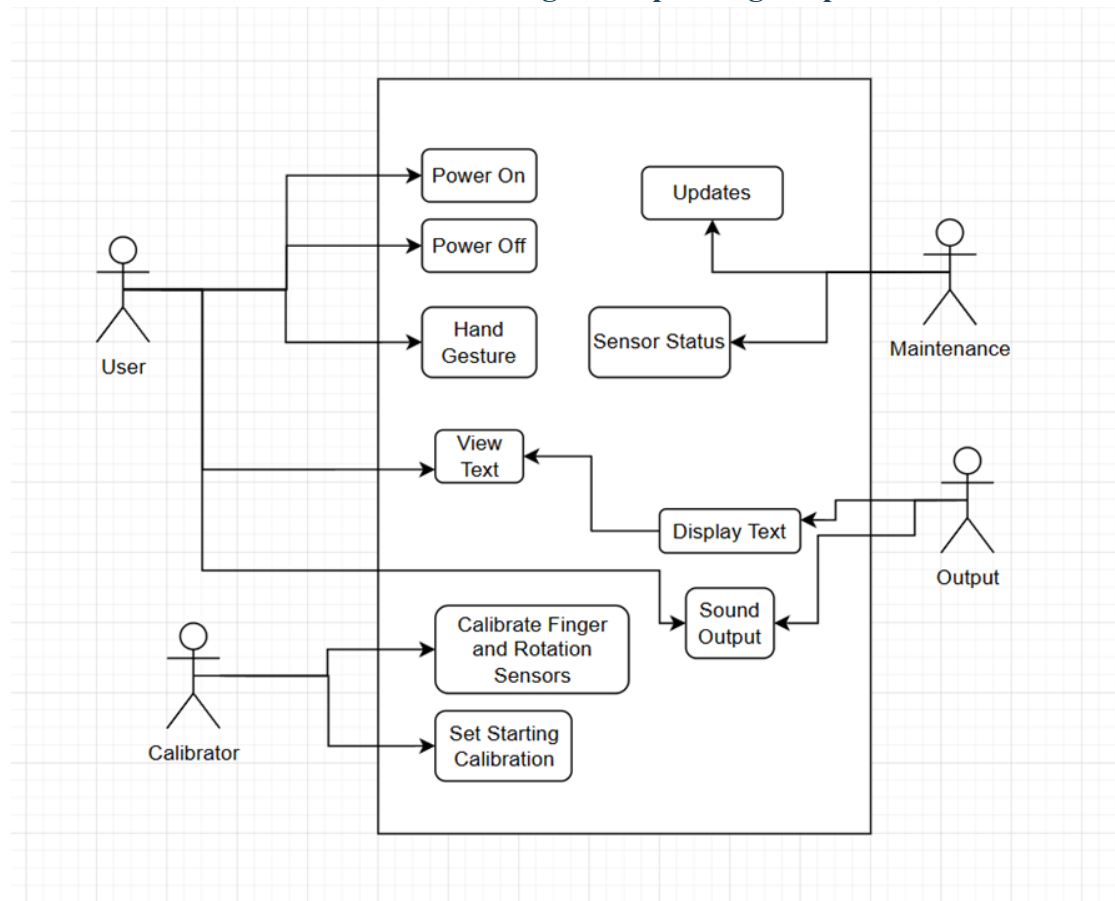
### Functional requirements (The "What"):

- Detect finger bending using flex sensors
- Classify gestures into corresponding letters, words, or phrases
- Display recognized gestures as text on a screen or into speech

### Non-Functional Requirements (The "How Well"):

- Process input data and generate output with minimal delay (soft real-time) within 1 second
- The interface will be simple and intuitive
- Be creative with signs - do not need to be exact like ASL (American Sign Language)

## UML use case diagram explaining the problem



## CRC cards explaining the modules of the solution and their responsibilities

### 1. Class: FlexSensor

- Responsibilities:
  - Measure finger bending and convert it into analog data
  - Send sensor data to the microcontroller
  - Calibrate raw input to ensure accurate readings
- Collaborators:
  - Microcontroller (for data collection)
  - GestureProcessor (uses sensor readings for classification)

## 2. Class: Microcontroller

- Responsibilities:
  - Collect and digitize signals from multiple FlexSensor objects
  - Preprocess data (filter noise, normalize values)
  - Communicate processed data to GestureProcessor
  - Manage timing for soft real-time response
- Collaborators:
  - FlexSensor (input source)
  - GestureProcessor (data consumer)
  - DisplayModule and AudioModule (for final output control)

## 3. Class: GestureProcessor

- Responsibilities:
  - Receive processed input data from the Microcontroller
  - Classify gestures
  - Translate recognized gestures into corresponding letters, words, or phrases
- Collaborators:
  - Microcontroller (data source)
  - TextOutput (for display)
  - AudioOutput (for speech conversion)

## 4. Class: TextOutput

- Responsibilities:
  - Display recognized gesture text on a screen
  - Format and update text output in real time
- Collaborators:
  - GestureProcessor (provides recognized text)
  - UserInterface (may include configuration/display settings)

## 5. Class: AudioOutput

- Responsibilities:
  - Convert recognized gestures or text into synthesized speech

- Maintain minimal delay between gesture recognition and speech output
- Collaborators:
  - GestureProcessor (provides recognized text)
  - UserInterface (controls volume, language, etc.)

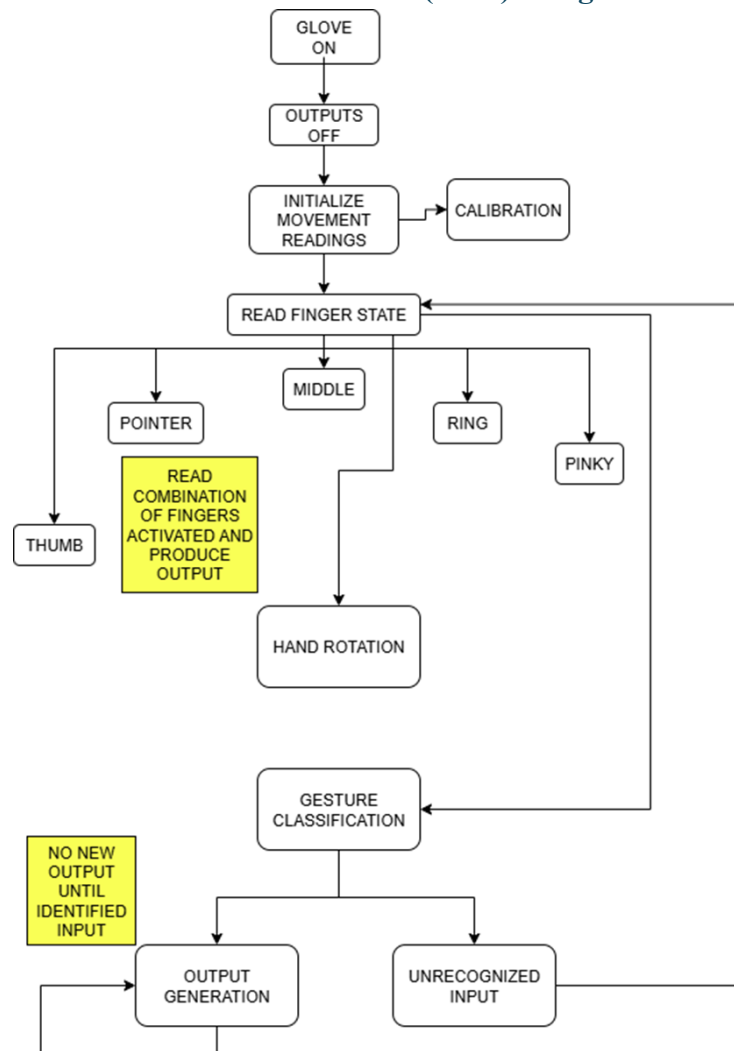
#### 6. Class: UserInterface

- Responsibilities:
  - Provide an intuitive and simple interface for users
  - Allow configuration (e.g., sensitivity, sound settings)
  - Display system status and feedback to the user
- Collaborators:
  - TextOutput
  - AudioOutput
  - Microcontroller (for sensor calibration commands)

#### 7. Class: SystemController

- Responsibilities:
  - Manage coordination between all system components
  - Monitor timing to maintain soft real-time constraints
  - Handle system startup, calibration, and shutdown sequences
- Collaborators:
  - Microcontroller
  - GestureProcessor
  - TextOutput
  - AudioOutput

## Finite State Machine (FSM) Design



## Component List

- Hardware:
  - Microcontroller: Arduino Nano
  - Sensors: 5x flex sensor
  - Power: 9V battery
  - Glove
  - Passive Components: 5x Resistors (10 kΩ)
  - Display/output module: 16x2 LCD
- Software:

- Arduino IDE / PlatformIO
- Wokwi Simulator (for initial testing)

# PHASE 2

## Understanding and Explanation of Hardware Components

For our prototype, we used **5 flex sensors** to resemble each finger on a human hand.

- **Input/Output:** Each flex sensor provides analog input. The analog values will be sent to the Arduino, where they (the values) are read and processed in real-time.
- **How it works:** The values from flex sensors will be changed when the sensor is bending. A large bend (such as 180 degrees) will increase the resistance, which gives changes in the voltage across the sensor.
- **Why it was chosen:** Flex sensors are thin, flexible, easy to mount on a glove, and provide consistent continuous values representing finger movement - ideal for detecting hand gestures.

The output device is a **16x2 LCD display**:

- **Input/Output:** values read and processed from the flex sensor through Arduino. It will display the correspondence text that matches the detected gesture on the LCD.
- **How it works:** After reading the analog values from the flex sensors, the Arduino runs logic to classify the gesture. The resulting text output (e.g., a letter or phrase) is sent to the LCD using digital control pins. The LCD then prints the message on its screen.
- **Why was it chosen:** It is useful when you want to debug/see if the output values match the expectation or not. When doing on breadboard, we can see the output on the Serial Monitor, but when we have the actual hardware, it is easier to look at

## Basic Functionality:

We successfully make the hardware work as expected:

- Reading analog values from flex sensors to Arduino
- The Arduino processes the signal to determine the hand gesture, finger position
- The results are displayed on the 16x2 LCD screen as text output

## Application to Final Project

For the final design, all hardware will be soldered/integrated into a glove. Each sensor will be aligned with a finger so that it can read the values directly based on the current hand/finger gesture.

We also plan to add a speaker with pre-recorded sound bites to correspond with the output on the display. This will match with real life situation where the opposite person can understand the hand gesture through hearing the explanation instead of watching text on LCD.

Also, we might need to buy the 9V battery as a power supply, LCD ribbon cable to get rid of the wires – make it clean, a glove to mount the sensors and components on.

### **Software Component – Codebase (30%)**

#### **1. Codebase Storage, collaboration and organized**

- a. Team shows how they are storing and sharing code and their ability to collaborate

We used a shared tinkercad file to work on the code and circuit for the prototype. This allowed us each to test the functionality of the device and add changes without needing to be physically connected to the circuit.

- b. Code is documented and contains references to external sources/library/author

```
#include <LiquidCrystal.h>
```

```
LiquidCrystal lcd_1(13,12,8,7,4,2);
```

```
int thumbPin = A0;
```

```
int pointerPin = A1;
```

```
int middlePin = A2;
```

```
int ringPin = A3;
```

```
int pinkyPin = A4;
```

```
int thumb, pointer, middle, ring, pinky;
```

```
int thumbLed = 10;
```

```
int pointerLed = 9;
```

```
int middleLed = 6;
```

```
int ringLed = 5;
```

```
int pinkyLed = 3;
```

```
int flex_min = 236;
```

```
int flex_max = 580;
```

```
int flexThreshold = 400;
```

```
void setup() {  
  pinMode(thumbLed,OUTPUT);  
  pinMode(pointerLed,OUTPUT);  
  pinMode(middleLed,OUTPUT);  
  pinMode(ringLed,OUTPUT);  
  pinMode(pinkyLed,OUTPUT);  
  lcd_1.begin(16,2);  
  lcd_1.print("Hello World!");  
  Serial.begin(9600);  
}
```

```
void loop() {  
  thumb = analogRead(thumbPin);  
  pointer = analogRead(pointerPin);  
  middle = analogRead(middlePin);  
  ring = analogRead(ringPin);  
  pinky = analogRead(pinkyPin);  
  
  Serial.println("Thumb: " + String(thumb));  
  Serial.println("Pointer: " + String(pointer));  
  Serial.println("Middle: " + String(middle));  
  Serial.println("Ring: " + String(ring));  
  Serial.println("Pinky: " + String(pinky));  
  Serial.println();
```

```
int thumbBrightness = map(thumb, flex_min, flex_max, 0, 255);  
int pointerBrightness = map(pointer, flex_min, flex_max, 0, 255);  
int middleBrightness = map(middle, flex_min, flex_max, 0, 255);  
int ringBrightness = map(ring, flex_min, flex_max, 0, 255);  
int pinkyBrightness = map(pinky, flex_min, flex_max, 0, 255);
```

```
thumbBrightness = constrain(thumbBrightness, 0, 255);  
pointerBrightness = constrain(pointerBrightness, 0, 255);  
middleBrightness = constrain(middleBrightness, 0, 255);  
ringBrightness = constrain(ringBrightness, 0, 255);  
pinkyBrightness = constrain(pinkyBrightness, 0, 255);
```



```

analogWrite(thumbLed, thumbBrightness);
analogWrite(pointerLed, pointerBrightness);
analogWrite(middleLed, middleBrightness);
analogWrite(ringLed, ringBrightness);
analogWrite(pinkyLed, pinkyBrightness);

bool t = thumb > flexThreshold;
bool p = pointer > flexThreshold;
bool m = middle > flexThreshold;
bool r = ring > flexThreshold;
bool k = pinky > flexThreshold;

// Debug: Print boolean values to see what's being detected
Serial.println("Bools - T:" + String(t) + " P:" + String(p) + " M:" + String(m) + " R:" + String(r)
+ " K:" + String(k));

// Rock on gesture (thumb and pinky bent, others straight)
if (!t && p && m && r && !k) {
  lcd_1.clear();
  lcd_1.setCursor(0, 0); // Changed from (0, 2) to (0, 0)
  lcd_1.print("Radical Dude!");
  Serial.println("Rad");
}
else if (t && !p && m && r && k) {
  lcd_1.clear();
  lcd_1.setCursor(0, 0); // Changed from (0, 2) to (0, 0)
  lcd_1.print("Pointing");
  Serial.println("p");
}
else if (!t && p && m && r && k) {
  lcd_1.clear();
  lcd_1.setCursor(0, 0); // Changed from (0, 2) to (0, 0)
  lcd_1.print("Good Job!");
  Serial.println("gg");
}
else {

```

```

lcd_1.clear();
lcd_1.setCursor(0, 0);
lcd_1.print("Hand Gesture");
lcd_1.setCursor(0, 1);
lcd_1.print("Ready");
Serial.println("None");
}

delay(300); // Single delay at the end
}

```

## 2. Program Flow

- a. Explanation of the main program loop(does not need to exist for demo)

Loop to check which fingers are bent and which fingers are not and display an output according to the conditions.

## 3. Testing Code and Demo Scripts

- a. Existence of test scripts or demo code to verify functionality of individual components.

Code above helped debug issues with our circuit. We use the serial monitor to calibrate the output of the flex sensors into a bent and not bent state. We monitored which fingers were inputting a bent signal, and which were not.

- b. Team has tested/demo every individual part of their project

We have 4 tests cases to begin with; standby mode, thumbs up, rock on, and pointer. These basic tests allowed us to determine functionality of the display and each finger sensor.

## 4. Progress on Final Codebase

- a. Explanation of how their current code base will help/lead to final project

We must build on our current codebase by adding more input combinations from the flex sensors, as well as adding code for playing sound clips through a speaker to match the output.

## Test Scripts and Testing Plan

### 1. Current Hardware Testing Verification

- a. Explanation of tests conducted to verify hardware functionality at this stage.

We tested each combination of finger positions to ensure the circuit was working as expected. So far the combinations that we coded all worked as expected when bending the required fingers and not bending the non-required fingers.

## **2. Future Hardware Testing Plan**

- a. Detailed plan for testing the final project's hardware functionality.

- **Test sound output and display output are synced**
- **Test expected output of finger movement is the same as actual output**
- **Ensure no outputs clash with each other**
- **Ensure glove works when placed on hand and powered on**
- **Ensure glove turns off when powered off**
- **Test speed of interpretation**

## **3. Current Software Testing Verification**

- a. Description of tests used to verify software components at this stage.

Using Tinkercad we were able to test the circuit functionality even before it was built. We ensured that the code ran as expected in simulation before uploading it to the hardware. Once hardware was built we ensured we got the same results that we did in the simulation.

## **4. Future Software Testing Plan**

- a. Plan for verifying final software functionality and software-hardware integration.

Continue to develop our software which uses Tinker cad or another qualified circuit simulation software. This allows us to have a full understanding of how to build our circuit before we use real hardware.

## **Photos of the Prototype**

### **1. Prototype Photos**

- a. Clear, labeled photos showing detailed views of the prototype.

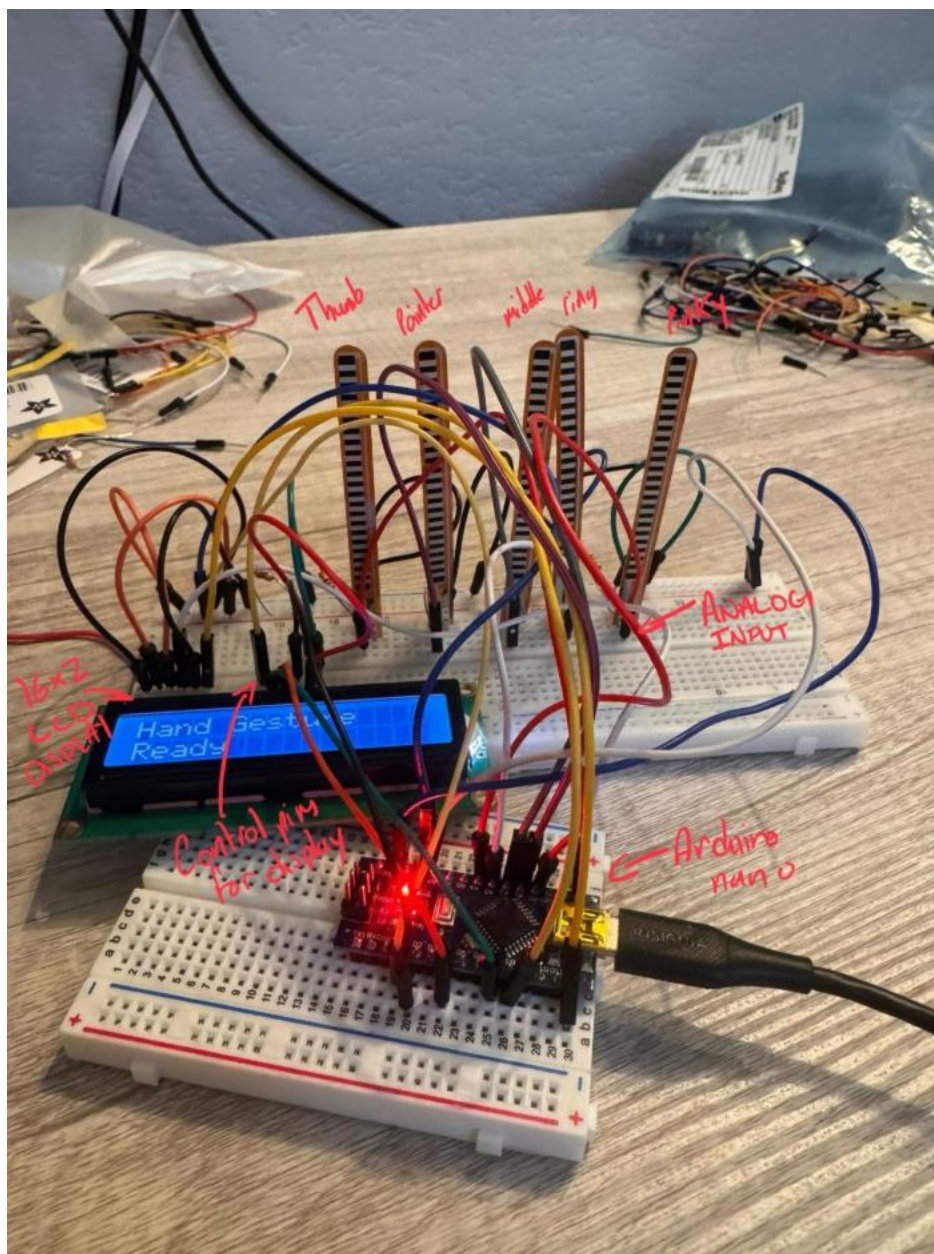


Figure 1

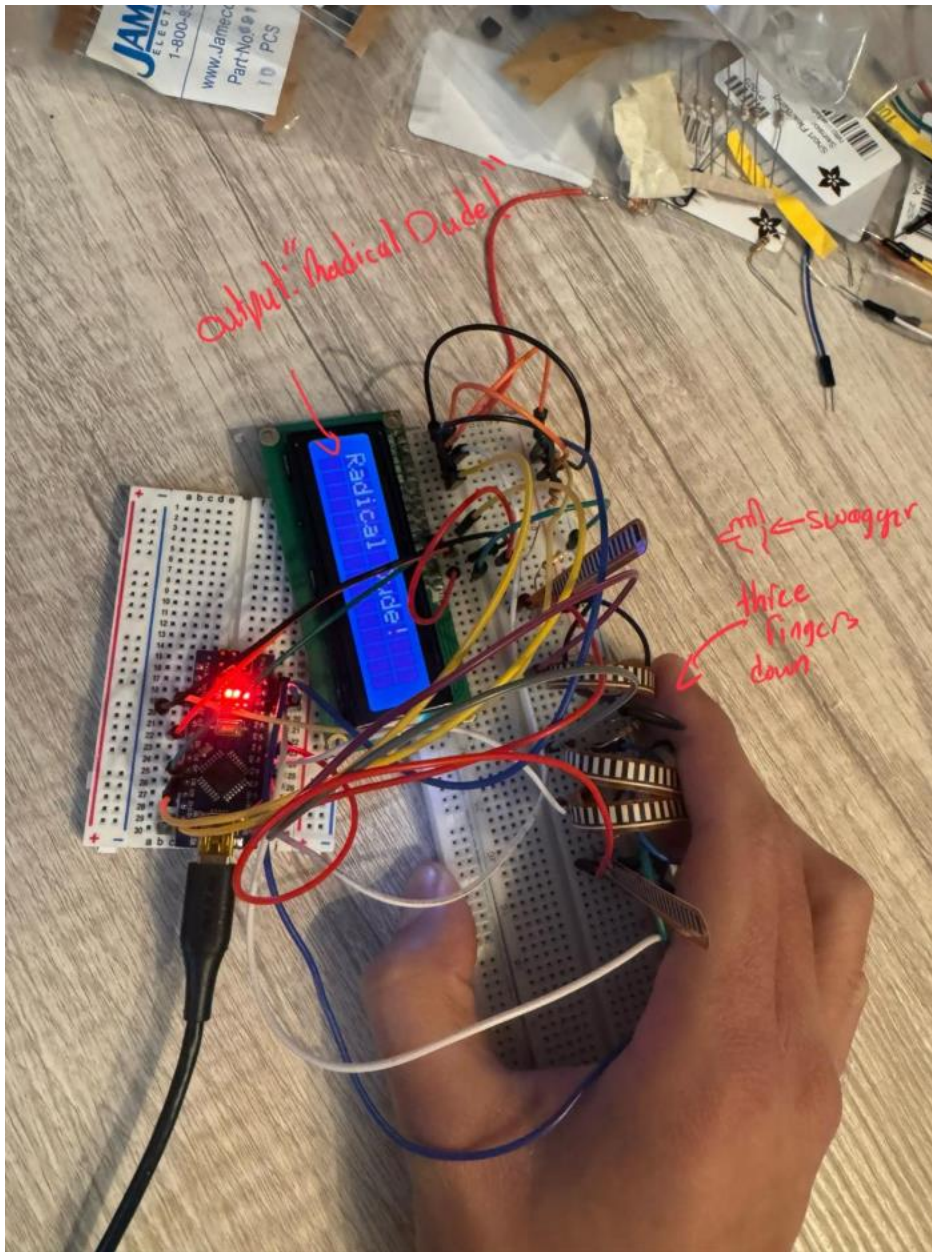


Figure 2

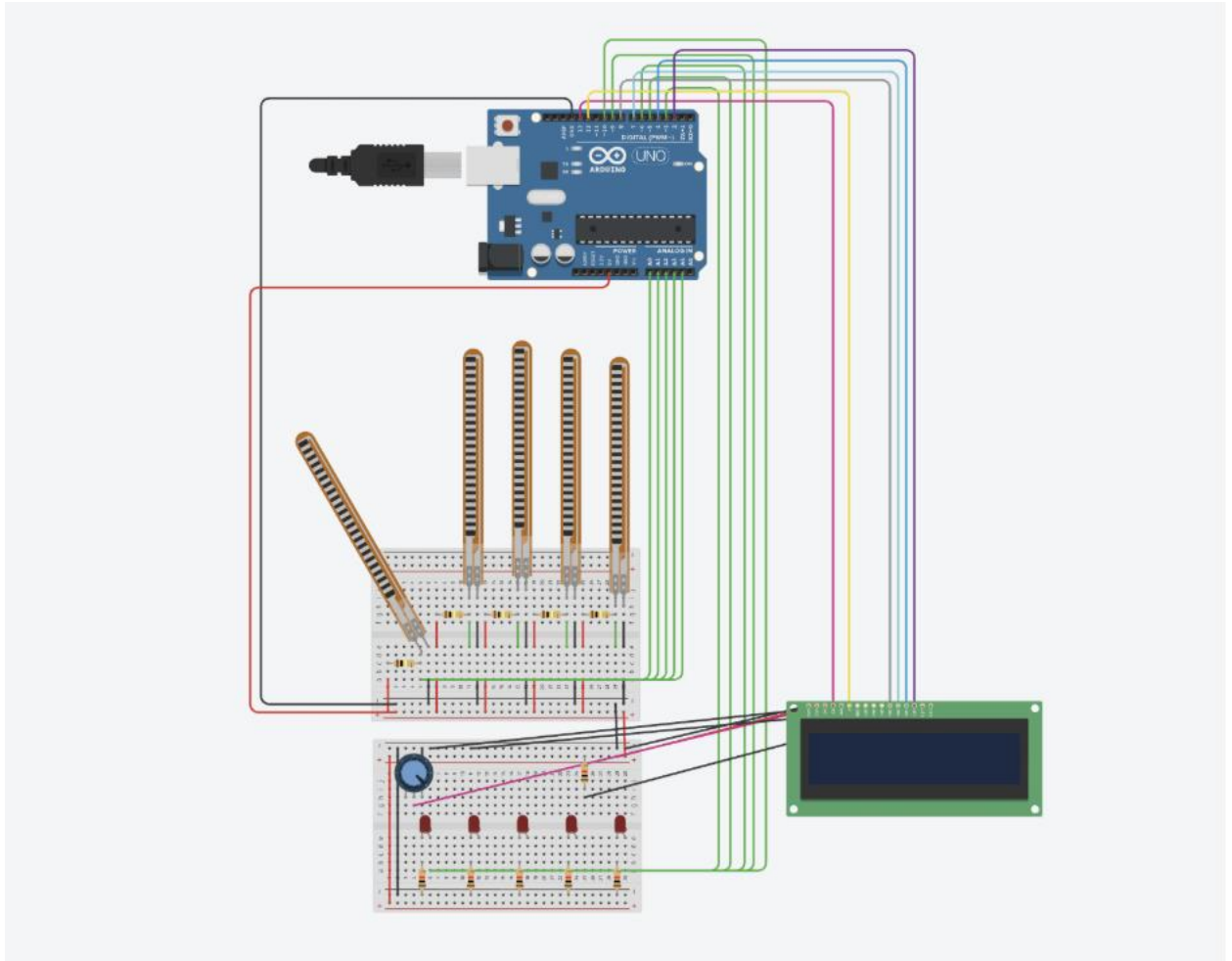


Figure 3

## 2. Photo Explanations

- a. Explanations for photos, describing the different parts, with labels for the different parts

Photos show circuit built on a bread board.

Figure 1 shows the circuit in idle state; no fingers are bent, and the program is looping checking for any finger bending.

Figure 2 shows the output of the system when the three fingers in the middle are bent forward. This hand gesture is the “rock on” hand emote and the display reads “Radical Dude!”

Figure 3 shows circuit built in simulation. This is what we used to initialize the design and modify the code when we were away from the circuit.



## Progress and Observations

### 1. Progress Overview

- a. Summary of the team's current progress.

Our prototype is working. We've successfully built a system where five flex sensors meant to be on a glove, send finger-position data to an Arduino. The Arduino recognizes specific gestures like a "thumbs up" or "rock on" sign and displays the correct message on an LCD screen. The core loop of reading, processing, and outputting is fully functional.

### 2. Setbacks and Solutions

- a. Explanation of challenges encountered and **strategies** for overcoming them.

Challenge: Calibrating the sensors and testing code without physical hardware. Cannot determine the right input for the sensor.

Solution: We used Tinkercad simulations to build and test the circuit and code collaboratively online first. We then used the Serial Monitor to fine-tune the sensor thresholds on the actual hardware.

Next Challenge: Transitioning from a messy breadboard prototype to a clean, wearable glove with audio.

Solution: We plan to solder the components, use a ribbon cable for the LCD, and add a battery. We will expand our current code to play matching sound clips from a speaker for each gesture.